System Verification and Validation Plan for DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

October 27, 2026

Revision History

Date	Version	Notes
Oct 26	1.0	Added system tests for authentication and task creation
Date 2	1.1	Notes

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 1

3 Plan **1**
- 3.1 Verification and Validation Team . . . 1
- 3.2 SRS Verification . . . 1
- 3.3 Design Verification . . . 1
- 3.4 Verification and Validation Plan Verification . . . 1
- 3.5 Implementation Verification . . . 1
- 3.6 Automated Testing and Verification Tools . . . 1
- 3.7 Software Validation . . . 1

4 System Tests **1**
- 4.1 Tests for Functional Requirements . . . 2
  - 4.1.1 Accounts and Authentication . . . 2
  - 4.1.2 Inference Task Creation . . . 7
- 4.2 Tests for Nonfunctional Requirements . . . 9
  - 4.2.1 Area of Testing1 . . . 9
  - 4.2.2 Area of Testing2 . . . 9
- 4.3 Traceability Between Test Cases and Requirements . . . 9

5 Unit Test Description **9**
- 5.1 Unit Testing Scope . . . 9
- 5.2 Tests for Functional Requirements . . . 9
  - 5.2.1 Module 1 . . . 9
  - 5.2.2 Module 2 . . . 10
- 5.3 Tests for Nonfunctional Requirements . . . 10
  - 5.3.1 Module ? . . . 10
  - 5.3.2 Module ? . . . 11
- 5.4 Traceability Between Test Cases and Modules . . . 11

6 Appendix **12**
6.1 Symbolic Parameters 12
6.2 Usability Survey Questions? 12

7 Appendix — Generative AI Use Disclosure **13**
7.1 AI Tools Used 13
7.2 How and Where Used 13
7.3 Verification of AI-Generated Content 13

8 Appendix — Reflection **14**

List of Tables

List of Figures

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

This document ...

2 General Information

2.1 Summary

2.2 Objectives

2.3 Custom Documents

2.4 Relevant Documentation

?

3 Plan

3.1 Verification and Validation Team

3.2 SRS Verification

3.3 Design Verification

3.4 Verification and Validation Plan Verification

3.5 Implementation Verification

3.6 Automated Testing and Verification Tools

3.7 Software Validation

4 System Tests

This section describes the tests that will be performed to verify that the entire integrated system meets the functional and non-functional requirements specified in the SRS. The system test cases below are be descrbed by the following characteristics:

- Control: Is this test manual or automatic?

- Initial State: What is the state of the system before the test is performed?
- Input: What is the input to the system for this test?
- Output: What is the expected output of the system for this test?
- Test Case Derivation: What FRs or NFRs does this test case cover?
- How test will be performed: What is the abstract procedure for performing the test?

4.1 Tests for Functional Requirements

4.1.1 Accounts and Authentication

These tests verify the account authentication requirements in the SRS, including authentication with a compatible Web3 wallet extension and an embedded wallet.

Authentication Using a Web3 Wallet Extension

1. ST-AUTH-EXT-1

   Control: Manual

   Initial State: The application is running in a supported browser on its public page. A compatible Web3 wallet extension is installed and active. Test profiles `EXT-USER-01` and `EXT-USER-02` are associated with wallet fixtures `EXT-WALLET-01` and `EXT-WALLET-02`; neither user is signed in.

   Input:

   - Connect `EXT-WALLET-01` and approve the verification-message signature requested by the application.
   - Connect `EXT-WALLET-02` and approve its verification-message signature.
   - Connect `EXT-WALLET-01` and reject the signature request.

   Output:

- For the approved signature from `EXT-WALLET-01`, the user is authenticated as `EXT-USER-01`, and that user's profile and authenticated features are available.
- For the approved signature from `EXT-WALLET-02`, the user is authenticated as `EXT-USER-02`, and that user's profile and authenticated features are available.
- The user remains unauthenticated, no profile or authenticated features become available.

Test Case Derivation: FR1 requires the authentication response to verify ownership of the connected wallet address.

How test will be performed: For each wallet, start from a signed-out session, connect the specified extension account, and approve the verification-message signature request. Confirm that `EXT-WALLET-01` opens `EXT-USER-01`'s profile and `EXT-WALLET-02` opens `EXT-USER-02`'s profile. Confirm that the user is granted access to the authenticated features. Repeat the process and reject the signature request; confirm that the user remains signed out and cannot access authenticated features.

2. ST-AUTH-EXT-2

   Control: Manual

   Initial State: The application is running on its public page in a supported browser. `EXT-WALLET-03` is available in a compatible browser extension and is not yet associated with a DAIIM user profile. The wallet owner is not signed in.

   Input:

   - Connect `EXT-WALLET-03` and approve the verification-message signature.
   - Sign out of the resulting account.
   - Reconnect `EXT-WALLET-03` and approve the verification-message signature again.

   Output:

- After the first approved signature, the user is authenticated and a profile is associated with `EXT-WALLET-03`. The system grants the user access to authenticated features.
- Sign-out ends the authenticated session; the user cannot access the profile or authenticated features while signed out.
- After the second approved signature, the same profile associated with `EXT-WALLET-03` is displayed. No second profile is created for the wallet. The system grants the user access to authenticated features.

Test Case Derivation: FR1 requires the system to verify ownership of the wallet address and associate that verified address with the same user profile on every successful authentication, including after sign-out. The second successful authentication must restore the profile created during the first sign-in.

How test will be performed: Record the number of DAIIM profiles associated with `EXT-WALLET-03`. Connect the wallet and approve the verification-message signature. Record the displayed profile identifier and confirm that the wallet is associated with that profile. Sign out and verify that the profile and authenticated features are no longer accessible. Sign in again with the same wallet and approve the signature. Verify that the displayed profile identifier matches the recorded identifier and that the number of profiles associated with the wallet has not increased.

Authentication Using an Embedded Wallet

1. ST-AUTH-EMB-1

   Control: Automatic

   Initial State: The application is running in a test environment with the embedded-wallet provider mocked. Identities `new.email@example.test`, `+15555550101`, and social-account subject `social-new-01` have no associated DAIIM account or embedded wallet. Configure the provider mock to return successful authentication and wallet creation for these identities, with distinct addresses `EMB-WALLET-NEW-01`, `EMB-WALLET-NEW-02`,

and `EMB-WALLET-NEW-03`, respectively. The malformed email `not-an-email` should be rejected by the system before the provider is called.

Input:

- Submit `new.email@example.test` using the email sign-in method.
- Submit `+15555550101` using the phone sign-in method.
- Submit social-account subject `social-new-01` using the supported social sign-in method.
- Submit malformed email `not-an-email` using the email sign-in method.

Output:

- For each of the three supported identities, the system creates one embedded wallet associated with the user's account, displays its corresponding address and a creation confirmation, authenticates the user, and makes the associated profile available.
- For `not-an-email`, the system reports an input error, displays no wallet-creation confirmation or address, and does not authenticate the user or create a wallet.

Test Case Derivation: FR2 requires successful creation for supported identity input after authentication, with the public address and confirmation as outputs. The system must reject malformed identity input before wallet creation. FR1 requires the successful wallet authentication to identify the user.

How test will be performed: For each valid test identity, call the corresponding embedded-wallet sign-in method with the specified email address, phone number, or social-account identity. For each successful case, compare the displayed address with the configured mock address, verify that the corresponding profile is authenticated, and confirm that exactly one wallet was created. For the malformed email, enter it using the email sign-in method and verify that the system reports an input error, does not call the provider, and does not create a wallet or authenticated session.

2. ST-AUTH-EMB-2

Control: Automatic

Initial State: The application is running in a test environment, and existing accounts `EMB-RETURN-01`, `EMB-RETURN-02`, and `EMB-RETURN-03` are signed out. Their embedded wallet addresses are respectively `EMB-WALLET-RETURN-01`, `EMB-WALLET-RETURN-02`, and `EMB-WALLET-RETURN-03`. Configure the embedded-wallet provider mock to return successful authentication responses with those address mappings. Configure it to return an authentication failure for `locked.user@example.test`, which has no associated account.

Input:

- Enter `returning.email@example.test` using the email sign-in method.
- Enter `+15555550102` using the phone sign-in method.
- Provide social-account subject `social-return-03` using the supported social sign-in method.
- Enter `locked.user@example.test` using the email sign-in method.

Output:

- For `returning.email@example.test`, the system authenticates `EMB-RETURN-01` and displays its existing profile and `EMB-WALLET-RETURN-01`.
- For `+15555550102`, the system authenticates `EMB-RETURN-02` and displays its existing profile and `EMB-WALLET-RETURN-02`.
- For social-account subject `social-return-03`, the system authenticates `EMB-RETURN-03` and displays its existing profile and `EMB-WALLET-RETURN-03`.
- For `locked.user@example.test`, the system reports authentication failure and does not create an authenticated session or grant access to a profile.

Test Case Derivation: FR1 requires each verified wallet address to remain associated with the same user profile after sign-out. Each successful response must restore the mapped existing profile rather than create another account. The failed authentication response does not verify wallet ownership, so it must not grant access.

How test will be performed: Call the application's embedded-wallet authentication entry point with the sign-in method and parameters from

each returning-user credentials. Configure the provider mock to return that fixture's existing wallet authentication response. Assert that each call returns an authenticated session for the user profile. Then call the same entry point with the sign-in method and identity parameters from the authentication-failure credentials. Assert that the call returns no authenticated session, grants no profile access and displays the appropriate error message.

4.1.2 Inference Task Creation

This test verifies the end-to-end task creation flow, including validation of the task definition, user authorization, and confirmation that the task-creation program instruction succeeds on the blockchain.

Create and Validate Inference Tasks

1. ST-TASK-CREATE-1

   Control: Automatic

   Initial State: The system is running on Solana Devnet. The requester wallet address fixture is authenticated, and the test harness is configured with its wallet signer and sufficient test SOL to pay transaction fees. The unique task names `Regression-Test-Task`, `Regression-Test-Min-Providers`, `Regression-Test-Invalid-Schema`, and `Rejected-Authorization-Task` are not already in use.

   Input:

   - A valid numerical regression task definition named `Regression-Test-Task`, including a description, numerical input and output schemas, an evaluation metric, a minimum provider count of 2 or more, reputation tolerance, acceptance period, and maximum prediction time.
   - The valid definition named `Regression-Test-Min-Providers`, with its minimum provider count changed to 1.
   - The valid definition named `Regression-Test-Invalid-Schema`, with an input or output schema containing non-numerical values incompatible with a numerical regression task.

- The valid definition named `Rejected-Authorization-Task`.

Output:

- For the valid definition with authorization approved, the task is created with the submitted definition, and the corresponding Solana Devnet transaction succeeds with the task-creation instruction.
- For the definition with fewer than 2 minimum providers, the system rejects the definition, and no task is created.
- For the definition with a schema incompatible with numerical regression, the system rejects the definition, and no task is created.
- For the valid definition with authorization rejected, no task is created and no successful task-creation transaction occurs.

Test Case Derivation: FR6 requires a task name, description, type, input schema, and output schema; the incompatible-schema case checks that the definition matches the numerical regression task. FR8 requires at least 2 providers, so a value of 1 must be rejected. FR5 permits authenticated users to create tasks, while FR4 requires wallet approval before an on-chain state change. Therefore, the valid task should be created only after approval, and rejecting authorization should leave it uncreated. FR50 and FR51 require a final transaction status and a transaction identifier that can be used to verify the successful instruction.

How test will be performed: Submit the valid definition to the task-creation flow and assert that the confirmation details match the submitted definition. Configure the test wallet signer to approve the authorization request. After confirmation, query Solana Devnet using the returned transaction identifier and assert that the transaction succeeded and contains the task-creation instruction. Query the application to confirm that `Regression-Test-Task` is available with the submitted definition. Submit the minimum-provider and invalid-schema definitions; assert that each is rejected before authorization and no task is created. Finally, submit the valid definition named `Rejected-Authorization-Task`, configure the signer to reject authorization, and assert that no task-creation instruction is submitted and no task is created.

4.2 Tests for Nonfunctional Requirements

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input/Condition:
   Output/Result:
   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input:
   Output:
   How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

5 Unit Test Description

5.1 Unit Testing Scope

5.2 Tests for Functional Requirements

5.2.1 Module 1

1. test-id1

Type:

Initial State:

Input:

Output:

Test Case Derivation:

How test will be performed:

2. test-id2

   Type:

   Initial State:

   Input:

   Output:

   Test Case Derivation:

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

5.3.1 Module ?

1. test-id1

   Type:

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

An AI assistant using the Copilot SDK in Visual Studio Code was used to assist with revisions to this document. The specific model used is GPT-6 Luna.

7.2 How and Where Used

The AI assistant was used to discuss, draft, and revise prose and requirements for the verification and validation work. In the VnV Plan, it helped structure system tests by functional area and draft and refine test cases for wallet authentication, embedded-wallet creation, returning-user authentication, and inference-task creation. Revisions focused on making inputs, expected outputs, test procedures, and requirement derivations clearer and more specific.

The assistant also helped revise related SRS content: clarifying support for embedded and browser-extension wallets, adding requirements for embedded-wallet creation and persistent wallet-to-profile association, and adding a conservative User Profile entry to the major data entities and data dictionary. The team directed the scope and reviewed or adjusted the proposed content.

7.3 Verification of AI-Generated Content

AI-assisted content in this document was checked against the SRS to confirm that each test case addresses stated requirements and does not introduce unapproved system behaviour or design decisions. Requirement references and the expected outcomes derived from them were checked against the corresponding SRS requirements and use cases. Test cases were reviewed for clear inputs, expected results, and procedures, and the LaTeX structure and cross-references were inspected after editing. Test fixtures, schema examples, and blockchain verification details are proposed examples and must be reconciled with the implemented system before execution. The team remains responsible for the accuracy, completeness, and consistency of this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?